

Understanding of Modern Software-Defined Networking

Phong Ngoc Hoang Nguyen
nhphong.nguyen@mail.utoronto.ca
University of Toronto

Abstract—This report summarizes an investigation of modern Software-Defined Networking (SDN), focusing on the evolution from the traditional network architecture, where the control plane and data plane are tightly coupled within individual network devices, to early SDN architecture that separate network control from packet forwarding, and ultimately to modern programmable network infrastructures deployed in large-scale data centers. The study examined ONOS, P4, Stratum, P4Runtime and Leaf-Spine Fabric. Particular attention was given to the separation of control and data planes, controller abstractions, programmable forwarding pipelines, and production-scale fabric architectures. The report concludes with some of the use cases, observations on the architectural tradeoffs and research challenges associated with programmable networks.

Index Terms—Computer Network, Software-Defined Network, SDN

I. INTRODUCTION

Started back in 1952, when IBM first manufactured large computer system and sold into the market as mainframe. It was bulky, large, and expensive machine that take up a room size. More importantly, every components within that mainframe is IBM's proprietary hardware, which means, a customer have to buy the whole vertically integrated solution from a single vendor regardless of the solution they want to have for their problem (software calculation, hardware design, finance...).

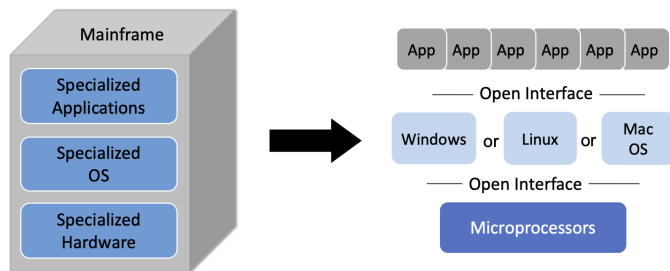


Fig. 1. Transformation from vertical market to horizontal marketplace with open interfaces at all level

The evolution began with the arrival of open-source operating systems and microprocessors, which helped transform the market into a horizontal one where innovation could happen at any layer.

SDN initiatives draw the similar inspiration by decoupling the control plane from the data plane. Decision-making logic is

decided by a external¹ logically centralized controller², using information from network operating systems enabled on top of the underlying switches. This abstraction simplifies switch's role, off-loading complex decision making logic (i.e., routing, path computation, ACL...) off-device and reducing the switch's primary task to packet forwarding. So they are referred to as "white box" or "bare-metal" switches³ which doesn't come with a fixed network OS, creating a hardware ecosystem for multiple operating system vendors.

OpenFlow is the pioneer protocol that support this disaggregation in the early stage of SDN. Although it was a main instrumental propelling SDN into the mainstream, OpenFlow now play a small role of today SDN because of an architectural limitation it imposed—namely, its fixed-function match-action pipeline. Modern infrastructure has largely transitioned toward fully programmable data planes driven by the P4 programming language, P4Runtime, and Stratum. The objective of this study is to explore this architectural evolution from early OpenFlow to current P4-based ecosystems deployed across large-scale hyperscale data centers.

II. BASIC ARCHITECTURE

The core philosophy of Software-Defined Networking is horizontal disaggregation across every layers of the networking stack. Rather than relying on a vertically integrated, propriety hardware appliances, SDN decompose them into abstract components that is independent from other components, allowing each of which being used interchangeably and innovated at each layer.

- **Local Network OS layer:** Physical switch hardware is decoupled from proprietary vendor software through bare-metal whitebox switches. Utilizing an Open Network Install Environment (ONIE) bootloader, the switch

¹In SDN terminology, a distinction exists between **external (global) network OS** and the **internal (local) network OS**. The External NOS refers to a distributed controller platform to maintain a centralized view of the network (ONOS section) while Internal OS refers to lightweight OS running directly on-switch to manage internal switch operation and communicate with the external network OS.

²While SDN control plane is logically centralized to provide a unified global view of network state and simplify network management, production deployment (introduced in ONOS section) are physically distributed for high availability and scalability

³"bare-metal" switches run a ONIE bootloader uses facilities in Linux kernel and BusyBox, defining an open source "install environment" for end users to install target network OS

can load any compatible local Network OS (e.g., Stratum, SONiC) to handle local hardware initialization and peripheral management.

- **Control-to-Data Communication Interface:** The control plane and data plane interact over well-defined consumer/provider API endpoints using standard protocol (e.g. OpenFlow, gRPC/P4Runtime). This bidirectional interface allows controller to program forwarding rules while collecting real-time telemetry, link states, metrics, and packet from the data plane.
- **Device Driver & Pipeline Layer:** This layer resolves hardware heterogeneity by splitting device control into low-level driver abstractions and pipeline-mapping logic. Analogous to a hardware SDK, the *device driver* handles low-level chip management, translate control protocol messages (e.g., OpenFlow or P4Runtime) directly into chip memory while absorbing vendor-specific behavior mismatches. Sitting above the driver, the *pipeliner* maps generic, high-level *flow objectives* into the exact multi-table forwarding pipeline required by the underlying ASIC, freeing higher-layer application logic from needing to know the switch’s internal table layout.
- **Data Plane layer:** Packet processing logic is completely decoupled from fixed-function hardware ASIC pipeline, allowing enterprise to implement custom header protocols on demand instead of waiting for switch vendor for the protocol implementation in their silicon chips. Using P4 language, packet parsing, header modifications, and forwarding behaviors can be programmed, updated, and recompiled directly on the hardware at runtime.

A. Single-switch perspective: The Software Stack

This includes an overview of a SDN stack with the involvement of a switch and controllers, highlight the top-to-bottom software stack being used in the SDN architecture. The main component include the Control App, Network OS, Switch OS, and P4 program—which used for the programmable switch. Figure 2 also introduced an exemplar of open-source software we used for this report (e.g. SD-Fabric, ONOS, Stratum), and protocol uses for communication between each layer.

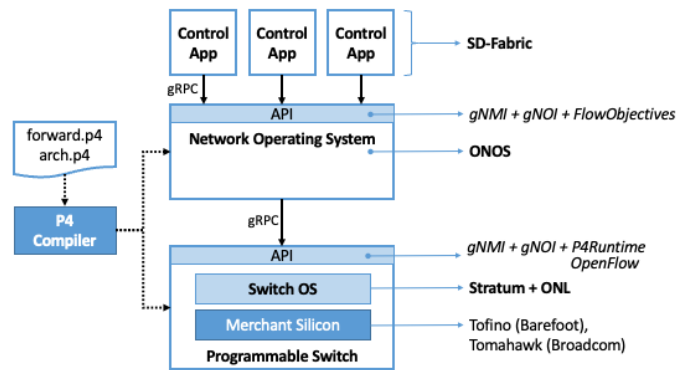


Fig. 2. Overall architecture of SDN stack with one switch

At the top of the stack, Control Applications communicate with the Network OS via gRPC over Northbound APIs. Rather than writing raw table entries, applications issue high-level, topology-agnostic *Intent*, or hardware-agnostic routing intents called *Flow Objectives* (e.g., policy constraints or forwarding goals). Concurrently, management interfaces such as gNMI (gRPC Network Management Interface) and gNOI (gRPC Network Operations Interface) are exposed on the Northbound interface, enabling control applications to query dynamic network state (e.g., topology graphs, link statuses) and execute operational lifecycle commands (e.g., rebooting or upgrading devices).

Below the control abstraction is the separation of the control plane and the data plane. The Network OS compiles high-level Flow Objectives into low-level, device-specific *Flow Rules*—concrete match-action entries targeted at explicit hardware table from the entire hardware pipeline. These rules are pushed down via gRPC using southbound protocols such as P4Runtime, OpenFlow, or gNMI. Upon receiving these instructions, the local Switch OS (e.g., Stratum, originally developed by Google) programs the entries directly into the hardware switch silicon⁴. Popular commercial implementations of this merchant silicon include Intel’s Barefoot Tofino and Broadcom’s Tomahawk chips.

B. Network-Wide perspective: The Entire Network Data Path

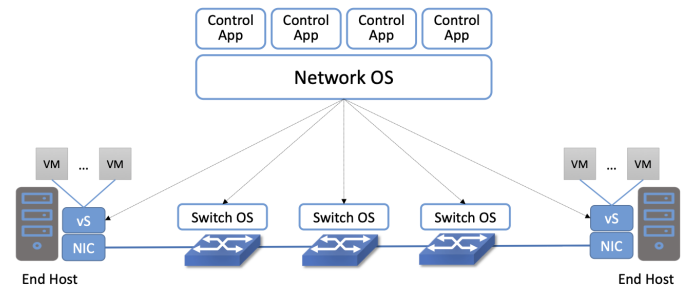


Fig. 3. End-End network view of SDN architecture

III. BARE-METAL SWITCHES AND P4 PROGRAMMABLE DATA PLANE

IV. SWITCH OS

V. NETWORK OS

VI. LEAF-SPINE FABRIC

VII. RESEARCH DIRECTION

⁴Switch silicon refers to a specialized integrated circuit (ASIC) designed for line-rate packet processing, switching, and hardware-accelerated QoS management.